Anumati — Product Requirements Document

Version 0.1 · **Date** 8 Oct 2026 · **Owner** Harshit Gupta · **Status** Hackathon build (AI Builder Cup 2026, BFSI track)

Anumati (अनुमति) — "permission / consent". A trust, risk and liability gateway for payments initiated by AI agents on UPI.

1. Problem

India is about to let software spend money. NPCI's Unified Agent Protocol (UAP) will let registered AI agents execute UPI payments without per-transaction approval, building on UPI Circle delegation (₹5,000 per transaction, ₹15,000 per month) and Reserve Pay blocks. Pine Labs' P3P is already live; Mastercard has demonstrated authenticated agentic transactions with Axis and RBL. Google's AP2 defines signed Intent / Cart / Payment Mandates that prove *authorisation*.

What none of this answers is the question an issuer, a customer, or an ombudsman will ask the first time an agent pays the wrong person:

Was this payment what the human actually authorised — or was the agent steered between the mandate and the rail, and by whom?

Three facts make this urgent:

1. **Mandates prove authorisation, not faithfulness.** A valid mandate plus a manipulated agent still yields a cryptographically "authorised" payment to a mule.
2. **Prompt injection is the dominant agentic risk** (OWASP maps it to 6 of its 10 agentic categories), and in commerce it arrives through content agents must read: listings, reviews, hidden page text.
3. **Legacy fraud engines are blind to it.** They profile human behaviour; an agent's deviation looks like a normal, authenticated, in-limit payment. Meanwhile mule-account networks are large and fast (27 lakh Layer-1 mule accounts in I4C's registry; 30–60 second intervention windows).

The liability framework for UAP has not been published. Anumati is a working proposal for what it should contain.

2. Users

User	Job to be done	Success looks like
Account holder (consumer delegating to an agent)	Give an agent narrow, understandable spending authority; be interrupted only when it matters	Writes one sentence; never sees a wrong payment; gets a one-tap prompt for the rare borderline case
Issuer / PSP risk team	Approve agent payments at UPI speed without owning unbounded liability	Deterministic controls they can audit, a semantic layer they can tune, a receipt per decision
Dispute / ombudsman analyst	Decide who bears a loss when an agent payment goes wrong	Replays a signed decision and gets a neutral attribution across user / agent provider / merchant / issuer
Agent provider	Get agents onto the rail with a predictable contract	Register once; clear verdict codes; evidence that exonerates a well-behaved agent

3. Goals and non-goals

Goals (v0, this build)

- G1. Compile natural-language mandates into enforceable, schema-validated guardrails, clamped to rail ceilings.
- G2. Dual-check every agent payment: deterministic policy + semantic audit (injection, intent drift).
- G3. Return a graduated verdict — APPROVE / STEP_UP / BLOCK — in well under a second.
- G4. Issue a signed, hash-chained Decision Receipt for every verdict, verifiable offline.
- G5. Provide a dispute workbench that replays a receipt and drafts an AP2-style liability attribution.
- G6. Ship a red-team sandbox and a synthetic attack suite with a caps-only baseline, so the control's value is measured, not asserted.

Non-goals (v0)

- Not a UPI PSP, not a wallet, no real money movement. The rail is simulated.
- Not a replacement for issuer transaction-monitoring; Anumati is an additional, agent-specific control.
- No agent identity PKI of its own — it consumes UAP/AP2 agent credentials when they exist (stubbed here).

- No human-fraud detection (social engineering of the *human* is out of scope).

4. Key scenarios

1. **Routine order** — agent buys the weekly basket at a mandated merchant → APPROVE in ~1 ms (heuristic) / <1.5 s (Gemini); receipt issued.
2. **Hidden-text redirect** — invisible `<div>` tells the agent the UPI id "migrated" to a mule VPA → BLOCK; findings: payee not verified for merchant, suspect-registry hit, injection evidence captured verbatim.
3. **Redirect to a fresh VPA** nobody has flagged yet → BLOCK on policy (unverified VPA for a known merchant + mandate forbids new payees) corroborated by semantic injection + drift.
4. **Review injection inflates cart** (12× ghee "mandatory") → STEP_UP; the human sees exactly why and declines in one tap.
5. **Cap splitting** — page asks for 3 × ₹1,800 → second payment BLOCKED as structuring.
6. **Dispute** — merchant claims a valid checkout was abandoned → analyst replays the receipt; attribution lands on the merchant/content provider with the injected text as evidence.

5. Functional requirements

5.1 Mandates

- FR-1 `POST /v1/mandates/compile` accepts free text; returns a *draft* mandate with `Guardrails` (per-txn cap, period cap + period, allowed merchants, payee policy, categories, step-up line, refundability, txn/day).
- FR-2 Compiler is Gemini with structured output; heuristic fallback; output is always clamped to `HARD_CAP_PER_TXN` / `HARD_CAP_PER_MONTH`.
- FR-3 Mandate becomes enforceable only after `POST /{id}/sign` (user confirmation; device-bound signature in production). Revocation is immediate.
- FR-4 Mandate shape is AP2-Intent-Mandate-compatible so a UAP/AP2 mandate can be ingested directly.

5.2 Agent registry

- FR-5 Agents must be registered (`POST /v1/agents`) and a mandate is bound to exactly one agent. Unregistered or mismatched agents are hard-blocked.

5.3 Decision engine

- FR-6 `POST /v1/payments/authorize` takes payee, amount, cart, agent reasoning trace and the content the agent consumed; returns a `Decision` with verdict, risk score, findings (each with code, weight, evidence), receipt id, optional step-up id.
- FR-7 **Policy checks (deterministic):** mandate state/expiry; agent binding; rail ceiling; per-txn cap; period cap (velocity); txn/day; structuring (≥2 payments ≥80% of cap to one

payee in 2h); payee allowlist via **verified merchant directory** (agent-declared merchant name is untrusted); suspect registry (I4C analogue); FRI risk; category scope (request + cart items); cart/amount integrity; quantity anomalies.
- FR-8 **Semantic audit (Gemini, heuristic fallback):** injection score with verbatim injected instructions; drift score with reasons. Runs even on hard failures because the receipt is dispute evidence.
- FR-9 Fusion: hard fail → BLOCK; else risk = f(policy Σ, injection, drift); ≥70 BLOCK, ≥40 or step-up rule → STEP_UP, else APPROVE. Thresholds are env-configurable.
- FR-10 Idempotency key support; p50 latency target < 50 ms excluding model call, < 1.5 s with Gemini 2.5 Flash.

5.4 Step-up

- FR-11 STEP_UP creates a `stepups` document; the console receives it via Firestore realtime listener (FCM push optional); `POST /v1/stepups/{id}/respond` settles or declines and annotates the receipt with the human's outcome (outside the signed body).

5.5 Receipts

- FR-12 Every decision issues a receipt: canonical JSON → SHA-256 → HMAC-SHA256 (key id); `prev_hash` chains receipts per mandate; `plain_language` summary; `liability_hints`.
- FR-13 `GET /v1/receipts/{id}` returns `signature_valid`; `POST /v1/receipts/verify` verifies any pasted receipt; `GET /v1/receipts/chain/{mandate}` verifies the chain.

5.6 Disputes

- FR-14 `POST /v1/disputes` replays the receipt and drafts an attribution (primary party, confidence, established facts, unresolved items, evidence needed). Gemini with heuristic fallback. Human resolves.

5.7 Sandbox & evaluation

- FR-15 `/sim/merchant/{scenario}` serves eight mock pages (clean + seven attacks); `/sim/run` drives a naive or Gemini-driven agent through the gateway.
- FR-16 `eval/run_eval.py` runs 40 synthetic cases (25 attack / 15 benign) against Anumati and a caps-only baseline and writes `RESULTS.md`.

6. Non-functional requirements

- **Explainability:** every verdict decomposes into named findings with evidence; no opaque score.
- **Fail-safe:** model unavailability degrades to heuristics, never to "approve by default" or "block everything".
- **Auditability:** receipts verifiable offline by any party; chain tamper-evident.
- **Latency:** decision path must fit inside UPI's interaction budget; Gemini call bounded by a 12 s timeout and skipped only when the mandate itself is missing.

- **Privacy:** no PII beyond VPAs and synthetic names; Firestore rules block all browser writes.
- **Deployability:** one script → Cloud Run (min-instances=1) + Firestore + Firebase Hosting; secrets in Secret Manager.

7. Google Cloud mapping

Component	Service	Why
Mandate compiler, semantic audit, dispute attribution	**Gemini 2.5 Flash** (Gemini API or Vertex AI)	Structured JSON output, low latency, long context for page content
Gateway API, policy engine, sandbox merchant + agent	**Cloud Run**	Stateless, scales to zero but pinned to 1 instance for judging
Mandates, agents, decisions, step-ups, receipts, disputes, registries	**Firestore**	Document model matches receipts; realtime listeners power step-up UX
Console: mandate authoring, step-up one-tap, ops feed, dispute workbench	**Firebase** Hosting + Auth (anonymous) + Firestore SDK; FCM optional	Hosting rewrites proxy `/v1/*` to Cloud Run so one origin serves everything
Signing key, Gemini key	**Secret Manager**	Rotatable; swap HMAC for Cloud KMS asymmetric keys without changing receipt format

8. Success metrics (hackathon)

- Sandbox: 8/8 scenarios land on the expected verdict live.
- Eval: attack catch rate ≥ 95%, benign false-flag rate ≤ 5%, versus caps-only baseline; results committed.
- Receipt chain verifies; a tampered receipt fails verification on stage.
- Demo video ≤ 3 minutes covering: mandate → clean approve → injected page block → step-up decline → dispute replay.

9. Risks & mitigations

Risk	Mitigation
Gemini quota/latency during judging	Heuristic fallback is always on; min-instances=1; 12 s model timeout
Frontier agent resists the injection, making the demo flat	Sandbox offers a *naive* scripted agent and a Gemini agent; both are honest about what they show
Heuristic thresholds overfit the suite	Suite is documented as a regression suite; Gemini run reported separately; thresholds are env-tunable
Scope creep	Mock merchant and agent are deliberately thin; no real rail integration in v0

10. Roadmap after the hackathon

1. Ingest real AP2 mandate VDCs and UAP agent credentials; verify signatures instead of stubs.
2. Replace mock registries with I4C Suspect Registry / DoT FRI feeds via the issuer's existing integrations.
3. Cloud KMS asymmetric receipt signing; receipt export to ombudsman CMS.
4. Issuer tuning console: per-segment thresholds, finding weights, shadow mode against live traffic.
5. Agent-provider SDK: a single `authorize()` call with trace capture.